哈尔滨工业大学（深圳）

软件构造导论
实验指导书

实验三 策略模式

2026 春

目录

1. 实验目的.....3
2. 实验环境.....3
3. 实验内容（2 学时）.....3
4. 实验步骤.....3
    4.1 弹道系统场景分析.....3
    4.2 策略模式代码实现.....4
5. 本次迭代开发目标.....7
6. 中期提交要求.....7

1. 实验目的

1. 深入理解策略模式的设计意图及结构
2. 结合案例，规范绘制策略模式的 UML 结构图
3. 掌握重构方法，通过代码实现并应用策略模式

2. 实验环境

1. IntelliJ IDEA 2025.3
2. OpenJDK 2

3. 实验内容

1. 深入理解策略模式的设计原理与结构，并结合实例绘制该模式的 UML 结构图；
2. 使用策略模式重构飞机弹道系统，实现不同机型的弹道发射逻辑，以及两种火力道具的加成效果；
3. 完成本次迭代开发清单的全部任务。

4. 实验步骤

4.1 弹道系统场景分析

在飞机大战游戏中，飞机的射击行为呈现出高度的多样性与动态性。英雄机弹道形态并非一成不变，而是会随着获取“火力道具”或“超级火力道具”在直射、散射、环射之间动态切换。敌机阵营的射击逻辑同样复杂：普通敌机不发射，精英与精锐敌机分别执行单排与双排直射，王牌敌机采用散射，而 Boss 敌机则拥有环射攻击。这些不同的攻击方式在子弹数量、飞行轨迹等方面均存在显著差异。

类型		弹道
普通敌机		不发射
精英敌机		单排直射
精锐敌机		双排直射
王牌敌机		散射
Boss敌机		环射
英雄机	无道具	直射
	火力道具	散射
	超级火力道具	环射

设计任务：

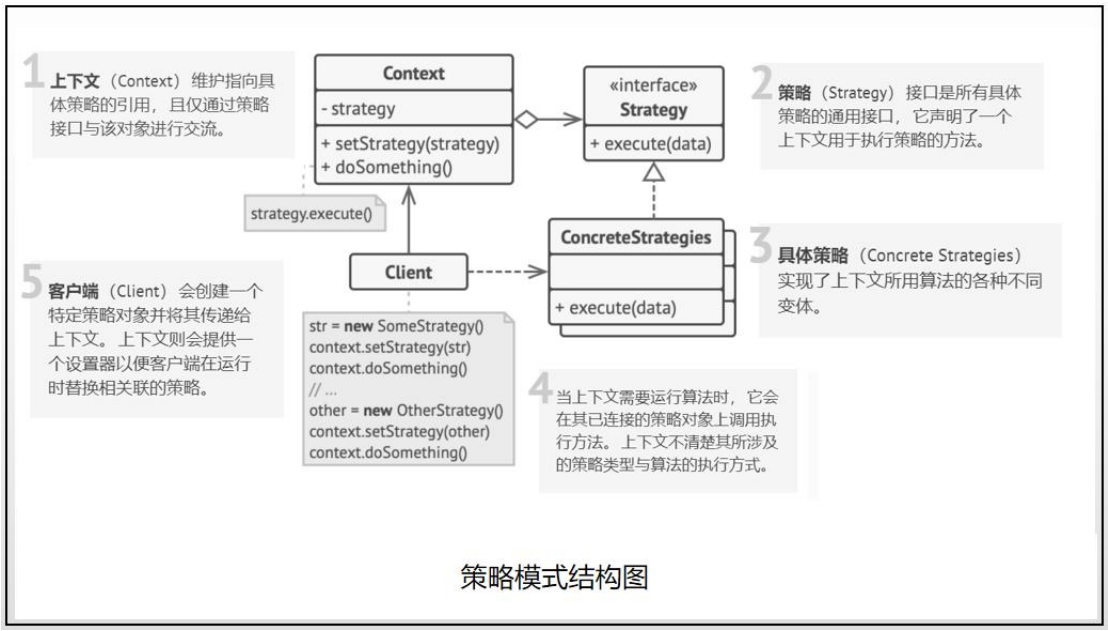
鉴于系统中存在多种可动态切换的弹道算法，且不同机型、道具效果对应不同的发射策略，为实现算法与业务逻辑的解耦、支持策略的灵活扩展与动态切换，请采用策略模式进行弹道系统的类设计与代码重构。

UML 建模：

绘制该模式的 UML 类图，图中需明确标注设计模式名称，并自定义合理的类名、属性名及方法名，以完整展示策略模式的实现结构。

策略模式

策略模式（Strategy Pattern）是一种行为设计模式， 它能让你定义一系列算法，并将每种算法分别放入独立的类中，以使算法的对象能够相互替换。



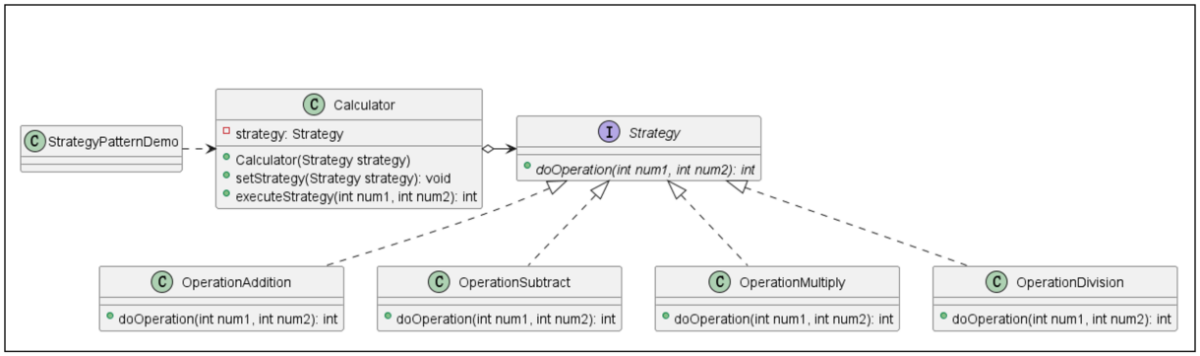
策略模式结构图

请参考以上 UML 结构图，绘制飞机大战中的策略模式（客户端不做要求）。

4.2 策略模式代码实现

假设我们想要实现一个计算器，使其支持加、减、乘、除四种运算。我们该如何运用策略模式来实现这一需求？

我们将创建一个策略接口（Strategy）以及四个实现该接口的具体策略类。计算器类（Calculator）将充当上下文（Context）角色，使用某一种具体策略。StrategyPatternDemo（作为客户端）将通过使用计算器与这些策略对象，来演示运行时替换策略时，Calculator 的行为是如何动态变化的。下图是将策略模式应用于该计算器问题的 UML 类图设计。



步骤 1：创建一个接口，充当抽象策略角色。

Strategy.java

```
public interface Strategy {
    int doOperation(int num1, int num2);
}
```

步骤 2：创建实现该接口的实体类，充当具体策略角色。

OperationAdd.java

```
public class OperationAdd implements Strategy{
    @Override
    public int doOperation(int num1, int num2) {
        return num1 + num2;
    }
}
```

OperationSubtract.java

```
public class OperationSubtract implements Strategy{
    @Override
    public int doOperation(int num1, int num2) {
        return num1 - num2;
    }
}
```

OperationMultiply.java

```
public class OperationMultiply implements Strategy{
    @Override
    public int doOperation(int num1, int num2) {
        return num1 * num2;
    }
}
```

OperationDivision.java

```
public class OperationDivision implements Strategy{
   @Override
   public int doOperation(int num1, int num2) {
        try {
```

```
            int num3 = num1 / num2;
        } catch (Exception e) {
            e.printStackTrace();
        }
        return num1 / num2;
    }
}
```

步骤 3：创建 Calculator 类，充当上下文（Context）角色。

Calculator.java

```
public class Calculator {
    private Strategy strategy;

    public Calculator (Strategy strategy){
        this.strategy = strategy;
    }

     public void setStrategy(Strategy strategy) {
        this.strategy = strategy;
    }

    public int executeStrategy(int num1, int num2){
        return strategy.doOperation(num1, num2);
    }
}
```

步骤 4：使用 StrategyPatternDemo 演示：当切换策略时，计算器的行为是如何变化的。

StrategyPatternDemo.java

```
public class StrategyPatternDemo {
    public static void main(String[] args) {

      Calculator calc= new Calculator(new OperationAddition());
      System.out.println("10 + 5 = " + calc.executeStrategy(10, 5));

      calc.setStrategy(new OperationSubtract());
      System.out.println("10 - 5 = " + calc.executeStrategy(10, 5));

      calc.setStrategy(new OperationMultiply());
      System.out.println("10 * 5 = " + calc.executeStrategy(10, 5));

      calc.setStrategy(new OperationDivision());
      System.out.println("10 / 5 = " + calc.executeStrategy(10, 5));
    }
}
```

步骤 5：执行程序，输出结果：

```
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2
```

输出结果各不相同。由此可见，尽管使用相同的代码结构，不同的策略会产生不同的计算结果。这就是多态：同一接口，不同行为。

5. 本次迭代开发目标

本次迭代任务清单（Checklist）：

（1）**UML 类图绘制：**按照规范绘制策略模式的 UML 结构类图；

（2）**设计模式应用：**完成代码重构，采用策略模式实现不同机型的弹道发射逻辑，以及两种火力道具的加成效果；

（3）**敌机生成：**游戏界面可正常显示 5 种敌机，按预设周期随机出现；当玩家分数达到指定阈值时，触发 Boss 敌机生成；

（4）**敌机移动：**Boss 敌机悬浮于界面上方，仅执行左右移动操作，不向下移动；

（5）**敌机攻击：**Boss 敌机采用环射弹道，单次同时发射 20 颗子弹；

（6）**道具生成：**Boss 敌机被击毁时，随机掉落 3 个道具（道具类型无限制）；

（7）**道具效果：**火力道具生效后，英雄机将当前弹道切换为散射弹道；超级火力道具生效后，英雄机将当前弹道切换为环射弹道；炸弹、冰冻道具生效时，在控制台输出对应提示信息即可。

注：本次实验未涉及多线程相关实现，因此弹道切换后暂无法自动恢复，该功能将在实验五中进一步完善。

6. 中期提交要求

✧ **提交网址**

提交至作业提交平台（截止日期参考平台发布），请使用统一身份认证登录，推荐使用 Chrome 浏览器。

作业平台入口： https://grader.hitsz.edu.cn/login?next=/courses

✧ **提交内容**

需提交项目压缩包（将整个项目打包为 zip 格式），压缩包内需包含完整代码及 UML 类图，确保文件完整可正常打开。